

Assignment: NoSQL

CSE 511: Data Processing at Scale

Introduction

RocksDB is an embeddable persistent key-value store for fast storage environments. It uses a log-structured database engine, written entirely in C++ for maximum performance. Over the years, RocksDB has become a popular choice for a wide range of applications, including large-scale data processing, real-time analytics, and high-throughput transaction processing. It is used by companies such as Facebook and LinkedIn to power their critical data-driven applications. From database storage engines such as MyRocks to application data caching to embedded workloads, RocksDB can be used for a variety of data needs. It provides basic operations such as opening and closing a database, reading and writing, to more advanced operations such as merging and compaction filters. For this assignment, we will be exploring the more basic functionalities of this Key-Value Store.

In the scope of this assignment, we will use `subreddits.csv` of the Reddit database from the previous assignments. The `subreddits.csv` is also available [here](#). Also, in this assignment, only high-level steps and expectations are mentioned. You are expected to explore the necessary requirements to solve the problem statement.

Step 0: Creating the setup for RocksDB (0 pts)

In the scope of this assignment, you **MUST** use RocksDB [7.10.2](#). Hence, the first and most important step is to familiarize yourself with RocksDB and create a working setup for RocksDB. No Docker containers will be provided for this assignment.

The RocksDB library provides a persistent key-value store. Keys and values are arbitrary byte arrays. The keys are ordered within the key-value store according to a user-specified comparator function, `rocksdb`. You can refer [here](#) for some help on installation.

Note: *If you are using M1/M2 setups, we advise you to work on a multi-architectural docker you can create on your own or provided in the previous assignments to avoid system-specific issues.*

Step 1: Loading the Data (25 pts)

[Template Here](#). RocksDB supports various write operations such as `Put`, `Merge`, and `WriteBatch`. These operations are optimized for high performance, and have features such as memtable, write-ahead log, and compaction that allow for efficient and persistent write operations, making RocksDB a popular choice for applications that require high write throughput and low latency.

What to implement?

You need to load the data from the subreddits.csv file into RocksDB. However, the CSV has multiple columns, and RocksDB is a key-value store. The data has to be modified in some way for things to work. We will take advantage of the ID for the data being unique for every row and the header of the CSV file for this.

To store the data, you need to follow the format of the key being “<id_value>_<column_name>” where id_value is the value of the column id for the row and column_name is the name of the column. The value for this key will be the data in the column. Refer to the example below for more details.

description	id
A CSE 511 example value	sp_23511

In this example, the key value pairs for the row will be:

Key: sp_23511_description value: A CSE 511 example value

Key: sp_23511_id value: sp_23511

Note: As shown in the example, make sure to have a key-value pair for the ID column as well.

Step 2: Basic Operations

Your task is to implement the following operations for RocksDB. Each operation carries equal weightage. You can find a basic explanation for some basic operations [here](#), which will help you implement this assignment. You can also explore some existing examples [here](#).

1. Read (25 pts)

Read operations in RocksDB involve fetching data from the database and providing it to the application. The database provides various read operations like `Get`, `MultiGet`, and `Iterator` to fetch data. The `Get` operation returns the value associated with the given key, while `MultiGet` returns multiple values for a set of keys.

What to implement?

You must implement `MultiGet` in this stage of the assignment. The criteria for what you need to return can be found in the template code.

2. Iterator (35 pts)

In RocksDB, an `iterator` is used to traverse through a subset of keys and values in a database. It allows for efficient sequential or random access to the data and supports

forward and backward iteration. Iterators can be used to perform operations like range queries, prefix scans, and finding the next/previous key. However, care must be taken to use iterators correctly and avoid undefined behaviour.

What to implement?

You must implement the Iterator in this stage of the assignment. The criteria for what you need to return can be found in the template code.

3. Delete (15 pts)

The `delete` operation in RocksDB involves marking a key-value pair as deleted rather than physically deleting the data from the database. This is because RocksDB uses an LSM (Log-Structured Merge) tree-based storage engine, which compacts data during background processes. The actual physical deletion of data is done during compaction. When a delete operation is performed, the key-value pair is marked with a special tombstone marker and will not be returned in future read operations.

What to implement?

You must implement Delete in this stage of the assignment. The criteria for what you need to return can be found in the template code.

Submission Requirements & Guidelines

- **Individual work:** This is an **individual assignment**; all work must be your own.
- **What to submit on Canvas:** Submit **exactly one** C++ file named `assignment-nosql.cc`.
- **Filename/extra files penalty:** Using a different filename or submitting additional files will incur a **30-point penalty**.

Submission Policies

- Late submissions will **absolutely not** be graded (unless you have verifiable proof of emergency). It is much better to submit partial work on time and get partial credit than to submit late for no credit.
- Every student needs to **work independently** on this exercise. We encourage high-level discussions among students to help each other understand the concepts and principles. However, a code-level discussion is prohibited, and plagiarism will directly lead to the failure of this course. We will use anti-plagiarism tools to detect violations of this policy.

Test Command:

Update Rocksdb directory in **Makefile** and then:

```
mkdir test
make
./tester
```

Grading Rubric (100 pts)

```
chang@chang-pc:~/docker-file/nosql$ ./tester
=====
NoSQL Assignment Grading Rubric
=====

Test 1: Create and Load KVS from CSV (25 points)
-----
[PASS] RocksDB database created successfully
[PASS] CSV data loaded correctly into database
[PASS] Multiple entries loaded (batch write implemented)
Score: 25/25

=====

Test 2: MultiGet Operation (25 points)
-----
[PASS] Correct number of results returned: 4
  Key: 2qhlr_display_name => Value: auto [CORRECT]
  Key: 2qhlk_display_name => Value: productivity [CORRECT]
  Key: 2qhla_display_name => Value: linux [CORRECT]
  Key: 2qhlb_display_name => Value: microsoft [CORRECT]
[PASS] All values match expected results
Score: 25/25

=====

Test 3: Range Iteration (35 points)
-----
[PASS] Iterator returned results
Number of results returned: 51
[PASS] Correct number of results in range (expected 51)
[PASS] Results contain only display_name values (no keys)
Sample results (first 5):
  1. bioinformatics
  2. Python
  3. xkcd
  4. u_Lovetofuck4321
  5. lolcats
Score: 35/35

=====

Test 4: Delete Operation (15 points)
-----
Key '2cneq_id' exists before deletion
Value: 2cneq
[PASS] Delete operation returned OK status
[PASS] Key successfully deleted (verified via Get)
Score: 15/15

=====

FINAL GRADING SUMMARY
=====
Test 1 - Create & Load KVS: 25 points
Test 2 - MultiGet:          25 points
Test 3 - Range Iteration:   35 points
Test 4 - Delete Operation:  15 points
-----
TOTAL SCORE: 100/100

Grade: A (Excellent)
=====
chang@chang-pc:~/docker-file/nosql$ █
```